

TDD

Il **Test-Driven Development** (TDD) è un approccio alla programmazione che mette i **test automatici** al centro del processo di sviluppo.

Prima si scrive un test che descrive un comportamento desiderato, poi si scrive il **codice minimo** per far passare quel test, e infine si migliora il design mantenendo i test verdi.

Questo metodo aiuta a scrivere codice **pulito, modulare e privo di bug**, perché ogni nuova funzionalità viene costruita partendo da un test che ne definisce i requisiti.

Cos'è il TDD

Il TDD si basa su un ciclo molto breve chiamato **Red → Green → Refactor**:

1. **Red**: scrivi un test che fallisce (perché la funzionalità non esiste ancora).
2. **Green**: scrivi il codice minimo per far passare il test.
3. **Refactor**: pulisci e migliora il codice, mantenendo tutti i test verdi.

Ogni ciclo aggiunge una piccola funzionalità ben testata, mantenendo il progetto stabile e comprensibile.

Il Ciclo TDD in pratica

Ecco un esempio molto semplice:

```
# test_calcolatrice.py
from calcolatrice import somma

def test_somma_due_numeri():
    risultato = somma(2, 3)
    assert risultato == 5
```

Questo test descrive il comportamento atteso: **la funzione `somma` deve restituire la somma di due numeri.**

Se ora eseguiamo `pytest`, il test fallirà, perché la funzione non esiste ancora.

Scriviamo il codice minimo per farlo passare:

```
# calcolatrice.py
def somma(a, b):
    return a + b
```

Ora il test passa (verde).

Abbiamo completato il ciclo Red–Green–Refactor con una singola funzionalità.

Cosa sono i test unitari

I **test unitari** verificano piccole parti isolate del codice — in genere **una singola funzione o classe** — per assicurarsi che si comporti come previsto.

Sono la base del TDD e vengono eseguiti automaticamente ad ogni modifica del codice.

Caratteristiche dei test unitari:

- sono **veloci** da eseguire;
- non dipendono da file, database o rete;
- descrivono chiaramente il comportamento atteso del codice.

Scrivere test con Pytest

In Python, il framework più diffuso per i test unitari è **pytest**.

Per iniziare, basta installarlo:

```
pip install pytest
```

Un test è semplicemente una **funzione che inizia con** `test_` e contiene delle **asserzioni** (`assert`).

Esempio:

```
# test_esempio.py
def test_esempio_base():
    assert (2 + 2) == 4
```

Esecuzione:

```
pytest -v
```

L'opzione `-v` mostra il nome di ogni test e il suo stato (verde = passato, rosso = fallito).

Struttura AAA dei test

Un test ben scritto segue la struttura **AAA** (Arrange – Act – Assert):

1. **Arrange (prepara)**: crea gli oggetti o i dati necessari per il test.
2. **Act (agisci)**: esegui l'azione da testare.
3. **Assert (verifica)**: controlla che il risultato sia quello atteso.

Esempio:

```
from calcolatrice import somma

def test_somma():
    # Arrange
    a, b = 2, 3

    # Act
    risultato = somma(a, b)

    # Assert
    assert risultato == 5
```

Esempio pratico: Stack implementato con TDD

Vogliamo implementare una struttura **Stack** (pila LIFO — Last In, First Out) usando il TDD.

Test 1 — Pop su stack vuoto

Partiamo da un comportamento semplice: se proviamo a estrarre da uno stack vuoto, deve generare un'eccezione.

```
# test_stack.py
import pytest
from stack import Stack

def test_pop_stack_vuoto():
    s = Stack()
    with pytest.raises(IndexError):
        s.pop()
```

Ora scriviamo il codice minimo per far passare il test:

```
# stack.py
class Stack:
    def pop(self):
        raise IndexError("Stack vuoto")
```

Test verde : il comportamento di errore è corretto.

Test 2 — Push e Pop di un elemento

Ora aggiungiamo la funzionalità di inserire e rimuovere un elemento.

```
def test_push_e_pop_un_elemento():
    s = Stack()
    s.push("ciao")
    assert s.pop() == "ciao"
```

E il codice per farlo passare:

```
class Stack:
    def __init__(self):
        self._items = []

    def push(self, item):
        self._items.append(item)

    def pop(self):
        if not self._items:
```

```
raise IndexError("Stack vuoto")
return self._items.pop()
```

Abbiamo aggiunto la logica minima per gestire uno stack reale, e i test sono ancora verdi.

Test 3 — Ordine LIFO

Infine, verifichiamo che lo stack rispetti l'ordine **Last In, First Out**.

```
def test_push_multipli_pop_in_ordine():
    s = Stack()
    s.push(1)
    s.push(2)
    s.push(3)

    assert s.pop() == 3
    assert s.pop() == 2
    assert s.pop() == 1
```

Tutti i test passano: il nostro stack è completo!

Possiamo ora fare un **refactoring** se necessario, ma mantenendo tutti i test verdi come garanzia di correttezza.

Perché il TDD è utile

- Ti obbliga a **pensare prima** al comportamento desiderato, non all'implementazione.
 - Ogni funzione o classe ha un **test di sicurezza** che ne garantisce il corretto funzionamento.
 - Puoi modificare il codice liberamente: se rompi qualcosa, un test fallirà subito.
 - I test diventano **documentazione viva** del tuo codice.
-

Buone pratiche

- Scrivi **un test alla volta** e fallo passare prima di scriverne altri.

- Dai nomi chiari e descrittivi, ad esempio `test_pop_su_stack_vuoto_lancia_errore`.
- Evita test troppo generici o duplicati.
- Esegui i test spesso (`pytest -v`) per avere feedback immediato.
- Non serve testare codice banale (come getter o costruttori vuoti). Concentrati sulla logica.

Pytest: asserzioni e test parametrizzati

Pytest semplifica molto la scrittura di test con dati multipli grazie al decoratore `@pytest.mark.parametrize`.

Esempio:

```
import pytest

@pytest.mark.parametrize("a, b, atteso", [
    (2, 3, 5),
    (1, -1, 0),
    (10, 5, 15)
])
def test_somma_parametrico(a, b, atteso):
    from calcolatrice import somma
    assert somma(a, b) == atteso
```

Questo test verrà eseguito tre volte con coppie di valori diverse, evitando codice duplicato.

Testare eccezioni con Pytest

Spesso vogliamo assicurarci che una funzione **sollevi un errore** in certe condizioni.

```
import pytest

def dividi(a, b):
    return a / b

def test_divisione_per_zero():
```

```
with pytest.raises(ZeroDivisionError):  
    dividi(10, 0)
```

Se l'eccezione non viene sollevata, il test fallisce.

Conclusione

Il **TDD** non è solo una tecnica di testing: è una **metodologia di progettazione**.

Scrivere prima i test aiuta a definire **cosa deve fare** il codice, non **come farlo**.

Questo porta a un design più pulito, meno bug e maggiore fiducia nelle modifiche future.

“Il codice è finito non quando non hai più nulla da aggiungere,
ma quando non hai più nulla da togliere.”

— Kent Beck